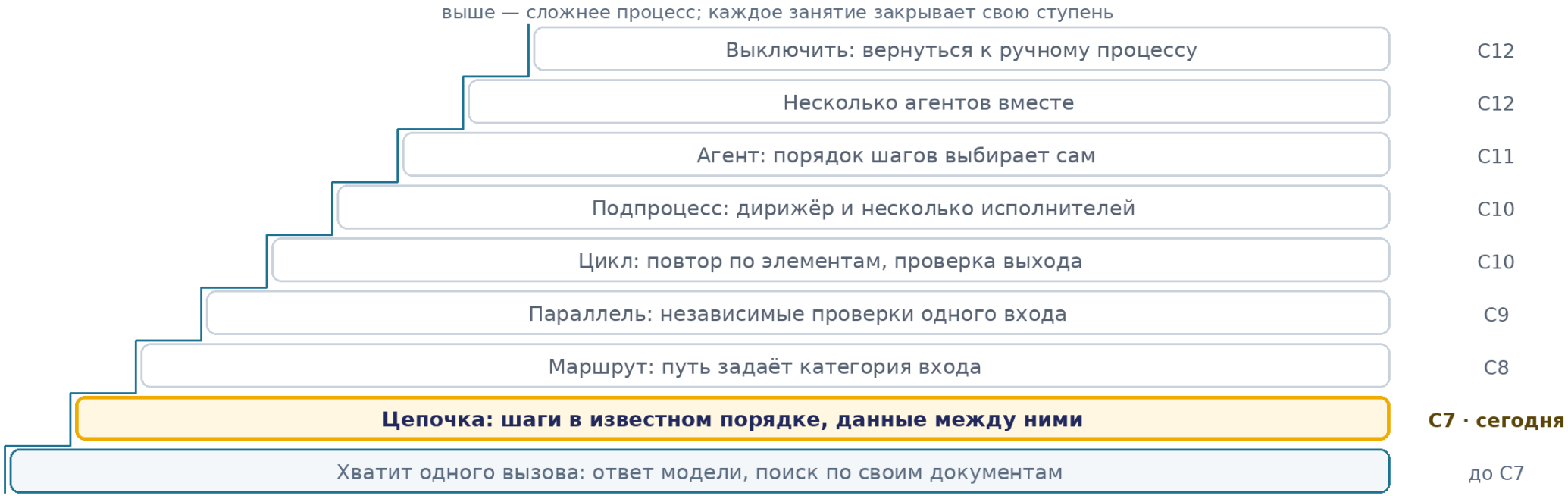
# Первый работающий процесс

Процессы на больших языковых моделях (LLM): занятие первое из шести.

# Куда мы идём: шесть занятий, девять ступеней




**Правило движения по ней, из Лекции 3: не поднимайся без требования, которого ступень ниже не закрывает. Это то же инженерное «делай простейшее, что работает», переведённое на язык процессов.**

**Сегодня мы закрываем вторую ступень снизу — цепочку. На ней заканчивается большинство реальных задач, и поэтому ей отведено целое занятие, а не десять минут.**

# Workflow и агент

**Workflow. Путь задан заранее. Модель — один из шагов: ей дают вход, берут выход, передают дальше. Поменять порядок может только тот, кто правит схему.**

**Агент. Путь определяется по ходу. Модель на каждом круге смотрит на промежуточный результат и выбирает следующее действие сама — из тех, что ей дали.**

**Выбор: задача предсказуема — workflow; последовательность шагов принципиально зависит от промежуточных результатов и задача оправдывает кратный рост стоимости — агент.**

**Пять схем workflow — все с заранее заданным путём:**

1. **Цепочка** — шаги друг за другом, выход одного на вход следующему. Сегодня.
2. **Маршрут** — первый шаг относит вход к классу, дальше у каждого класса свой путь.
3. **Параллель** — независимые подзадачи сразу, результат сводится детерминированно.
4. **Дирижёр и исполнители** — центральный шаг делит задачу в заданных рамках и раздаёт.
5. **Эталон и переделка** — один делает, другой оценивает, по кругу до порога.

**Большинство надёжных продакшн-систем — это workflow с предопределёнными путями, а не полностью динамические агенты.**

# Надёжность и цена: четыре числа

| 95% НА ШАГ | 60% НА ДЕСЯТИ ШАГАХ |
|---|---|
| **90%** на шаг | **35%** на десяти шагах |
| одиночный агент | **4×** токенов к обычному чату |
| мульти-агент | **15×** токенов к обычному чату |
| рой равноправных агентов | ошибка растёт в **17,2×** |
| один координатор | ошибка растёт в **4,4×** |

**Первые две строки. Если сбой любого шага портит результат, надёжность цепочки — надёжность шага в степени числа шагов. «Каждый шаг почти всегда работает, значит и цепочка почти всегда» — математически ложно. Отсюда два правила, которые мы применим руками: цепочки держат короткими, между шагами ставят проверки.**

**Вторые две. Мульти-агентные системы работают во многом потому, что помогают потратить на задачу достаточно токенов. Не магия координации, а объём.**

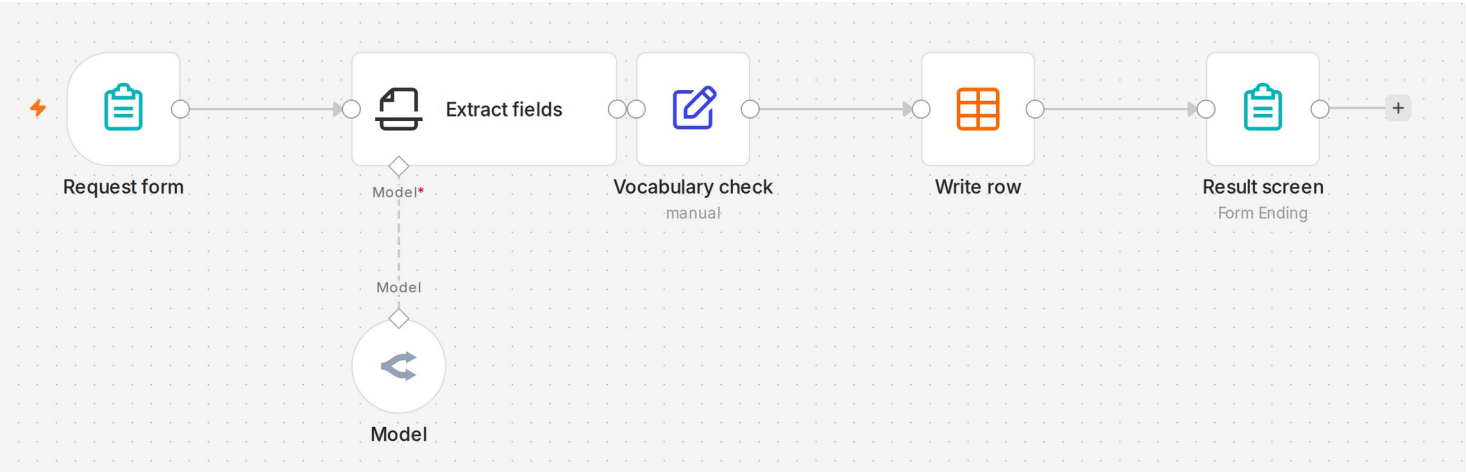
**Последние две. При одном и том же числе агентов дирижёр надёжнее роя — обратное тому, что обычно предполагают. Начинай с одного сильного: команды месяцами строили мульти-агентные схемы и обнаруживали, что улучшенный запрос к одному даёт то же самое.**

# Классы инструментов

| КЛАСС ЗАДАЧИ | ПРИМЕР | СИСТЕМЫ |
|---|---|---|
| **Перенос между системами** — возьми оттуда, преобразуй, положи сюда, уведоми | заявка с формы → строка в таблице → письмо менеджеру | **n8n** · Activepieces · Make · Zapier |
| **Задача агенту** — дай инструменты и проверки, поручи задачу | разобраться в обращении, сходить в три системы, предложить ответ | sim · Agent Builder у крупных поставщиков |
| **Приложение на модели** — цепочка вызовов, поиск по документам, память диалога | чат по корпоративной базе знаний | LangFlow · Dify |

**Запуск по расписанию и по событию есть у первого и второго класса. У третьего — только снаружи, вызовом или из чата.**

# Устройство n8n




**Холст и узлы — процесс не текст, а схема. Узел — шаг, щёлкнул — открылась карточка. Между правкой и проверкой нет ни сборки, ни развёртывания.**

**Учётные данные — отдельно от сценария. В файле процесса ключей нет: файл можно отдать студенту, а при ротации ключ меняется в одном месте.**

**Больше 400 интеграций из коробки — «взять из Таблиц и положить в Телеграм» — два узла, а не два дня с чужой документацией.**

**Форма и чат встроены — веб-страница с проверкой ввода и экраном результата без фронтенда и хостинга.**

**Журнал прогонов — каждое исполнение хранится целиком: видно, что пришло на вход любого узла и что он отдал.**

# Карта занятия: четыре кейса

| КЕЙС | ЧТО СОБИРАЕМ | ЧТО ВЫГЛЯДИТ РАБОЧИМ | ЧЕМ ОКАЗАЛОСЬ |
|---|---|---|---|
| **1. Классификация и сохранение писем** | форма → модель → сверка → таблица → экран | заявка доезжает до таблицы | |
| **2. Заявки с двумя темами** | справочник и то, что в него не влезло | клетка заполнена, значение законное | |
| **3. Отказ модели** | повтор попыток и ветка отказа | прогон зелёный, ошибок в журнале нет | |
| **4. Тема определена неверно** | усиленный запрос и отметка ошибки человеком | значение из справочника, прогон зелёный | |

**Третья колонка — то, что вы увидите своими глазами и сочтёте успехом. Четвёртую заполним по одной строке за кейс и вернёмся к ней в конце.**

# Кейс 1. Классификация и сохранение писем

*«Форма-то есть. Их всё равно потом руками в таблицу перекладывают.»*

# Текущий процесс

**Учебный центр, на сайте форма записи: имя, контакт и свободный текст «что нужно». Форма работает третий год, заявок двадцать в день. Дальше с ними работает один человек — администратор Ольга.**

1. Человек пишет своими словами: «хочу на курс по аналитике, перезвоните сегодня после шести», «можно перенести занятие со вторника», «выставьте счёт на организацию».
2. Ольга открывает каждое письмо, читает, понимает, о чём оно, и переносит в таблицу — отдельно контакт, отдельно тема, отдельно срочность.
3. По этим двум столбцам вечером раскладывают заявки: кому звонить сегодня, кому на неделе, что отдать в бухгалтерию.
4. В пятницу заявок больше, а времени меньше. Появляется «потом разберу» — и что-то не разбирается.

**Чем платят за ошибку в теме и срочности. Заявку смотрит не тот человек → она идёт лишними передачами из рук в руки → обработка задерживается минимум на день. А каждый день просрочки в ответе снижает вероятность, что человек всё-таки купит.**

**Сорок минут в день — меньшая часть цены. К двадцатому письму подряд работа делается механически: не потому, что Ольга плохая, а потому что двадцатое похоже на пятое.**

# Что делать с потоком писем

**Как своевременно и качественно разбирать 200 писем в день?**

| | | | |
|---|---|---|---|
| **добавить сотрудников** | **алгоритмическая NLP-классификация по ключевым словам** | **один шаг модели** | **шаг модели + алгоритмическая сверка со справочником, что не вошло — в «не определено»** |

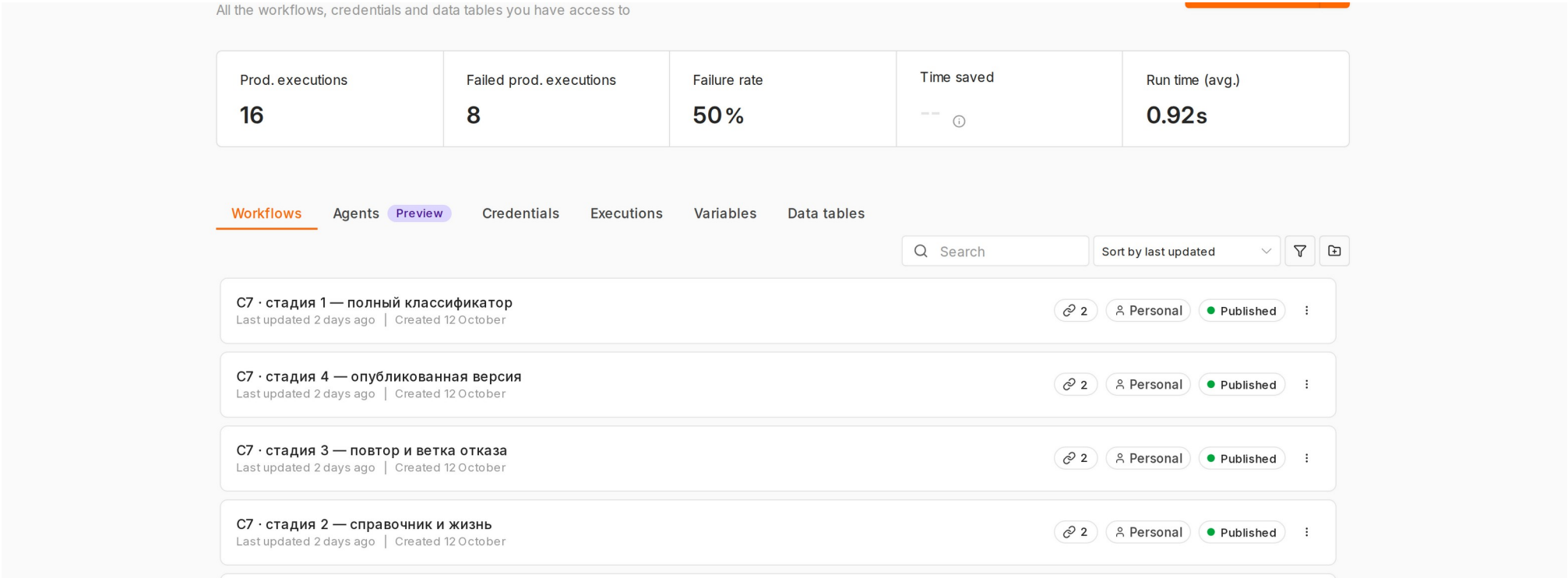
# Что делает каждый способ

| СПОСОБ | ТОЧНОСТЬ НА ЖИВОМ ПОТОКЕ | ЧТО ДЕЛАЕТ | ЧЕГО НЕ ДЕЛАЕТ |
|---|---|---|---|
| **добавить сотрудников** | **≈85%** — замер по журналам трёх служб поддержки | видит контекст целиком и замечает то, чего в списке нет вовсе | линейно дорожает: двести писем — это уже не один человек. И согласованность падает к концу смены |
| **алгоритмическая NLP-классификация** | **53%** на семи категориях, причём на двух из них — **0%** | предсказуема, дёшева и отлаживается глазами: одно слово всегда даёт одну тему | ломается на живом языке: «а можно к вам», «счёт на ООО» — ни одного ключевого слова. Правил становится много, и они начинают противоречить друг другу |
| **один шаг модели** | **75–80%** на семи категориях; на живом потоке бывает и ниже обученной модели | работает **без размеченной истории** — на описании категорий словами | не обещает формат: в клетке окажется «запись», «обучение», «Аналитика.» — каждый ответ разумный, а столбец негодный |
| **шаг модели + сверка со справочником** | смысловая та же, что у шага модели, **плюс 100% значений из справочника** | значение в таблице **всегда** из списка, пустых клеток не бывает; детерминированная сверка — её поведение известно заранее | смысловую ошибку не чинит: уверенно неверная тема пройдёт сверку, потому что она из списка |

**Поворот. Первые два ломаются громко: правило не нашло слово — клетка пустая, это видно. Модель ломается тихо: в клетке правдоподобное слово, которого нет в вашем списке, прогон зелёный, и ни одна проверка формата этого не поймает. Громкий отказ дешёвый, тихий дорогой.**

**Почему 100% из справочника стоит отдельного узла. Столбец читает не человек, а следующий шаг: сортировка, маршрутизация, отчёт. Человек поймёт «Аналитика.» и «аналитика» как одно; система — как два разных значения, и одно из них просто не найдётся фильтром.**

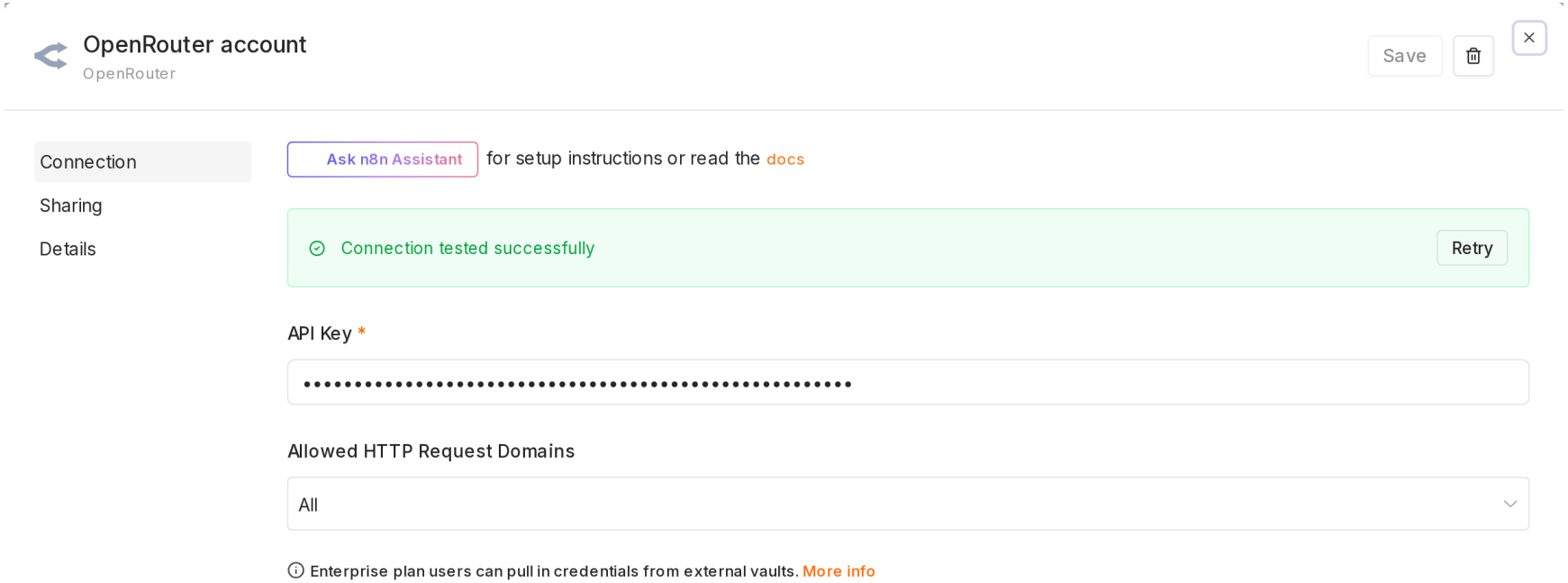
# Обзор: где что лежит




**Процессы — список того, что собрано. У каждого видно, опубликован он или нет, кто владелец и когда правили последний раз. Четыре строки на снимке — четыре стадии процесса этого занятия.**

**Счётчики сверху читаются буквально: сколько прогонов прошло в бою, сколько из них упало, какая доля и сколько в среднем занимает прогон. Половина падений на снимке — наша: стенд ломали специально, и в третьем кейсе мы это повторим.**

# Учётные данные

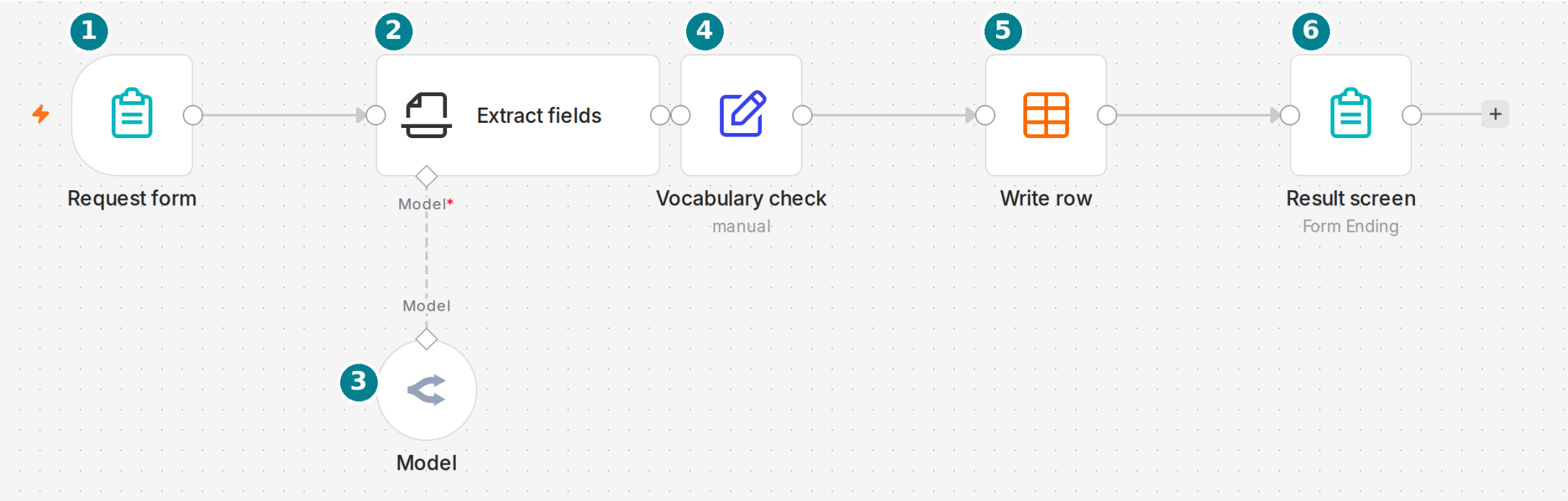



**Соединение проверяется здесь же. Зелёная строка на снимке — это не «сохранено», а «мы сходили в сервис и он ответил». Проверка до первого прогона, а не вместо него.**

**Ключ скрыт и после сохранения не показывается — даже владельцу. Посмотреть нельзя, можно только заменить.**

**В файле процесса ключа нет — только имя учётных данных. Поэтому процесс можно выгрузить и отдать: получивший подставит свой ключ. На этом построено домашнее задание.**

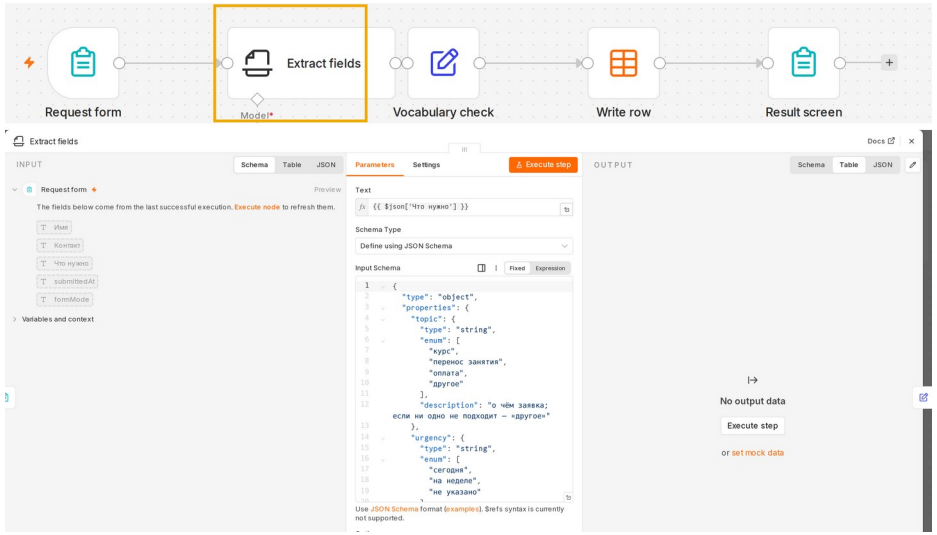
# Процесс целиком




1 форма — вход процесса · 2 извлечение — достаёт значения из текста · 3 модель — второй узел шага
4 сверка со списком · 5 запись строки · 6 экран результата — печатает всю дорогу

**Это настоящий холст n8n, а не схема. Дальше мы откроем два узла из шести и посмотрим, что в них настроено.**

# Настройка узла извлечения



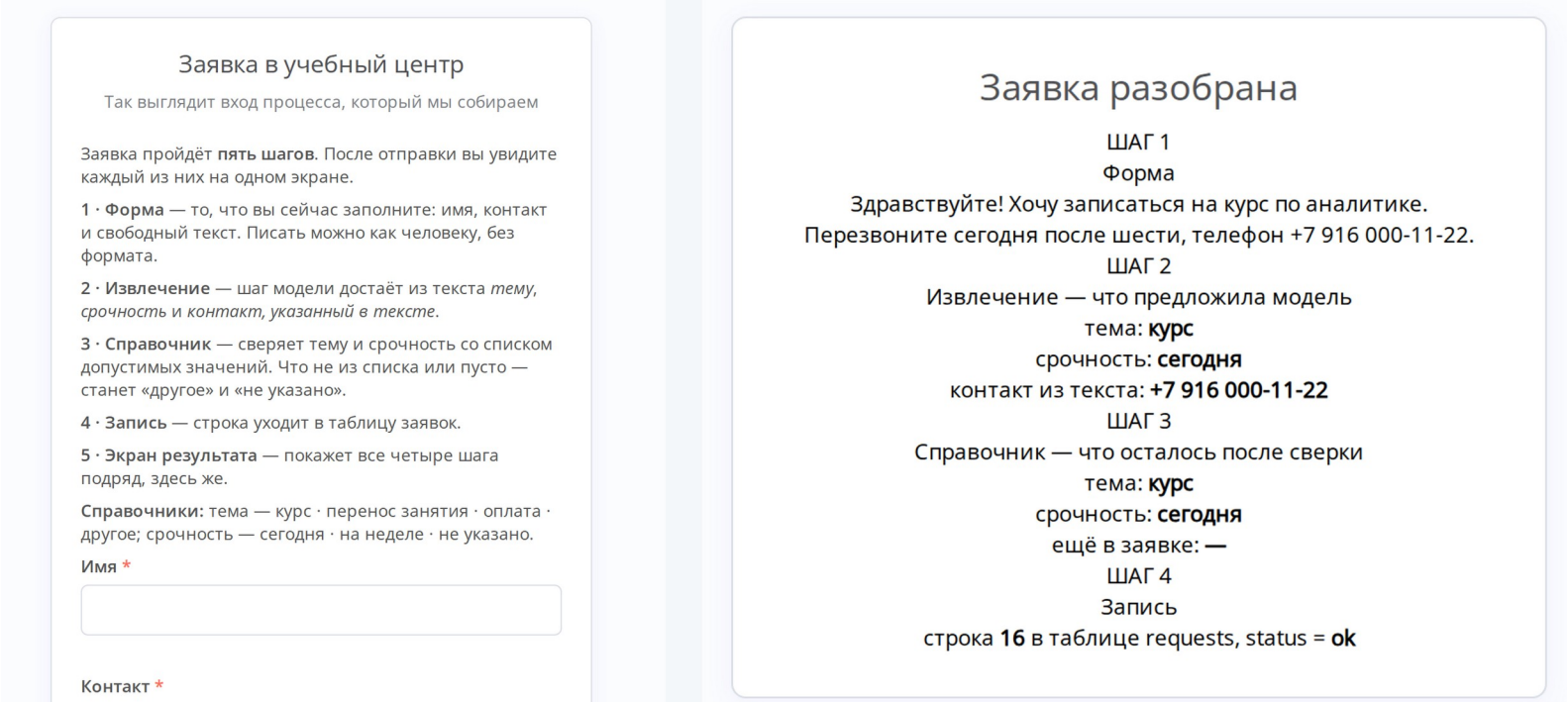

**Text — что читаем. Указываем одно поле формы: свободный текст заявки. Не всё письмо целиком, а именно его.**

**Схема — какие значения разрешены. Для темы перечисляем четыре допустимых значения, для срочности три. Модель выбирает из перечисленного, а не придумывает формулировку. Рядом с каждым полем — описание словами: по нему модель и понимает, что такое «срочность».**

**Обязательность. Тема и срочность помечены обязательными — пустыми не придут. Контакт из текста не помечен: его в заявке может не быть.**

**Сверка — отдельный узел. Обычный «задать поля»: если пришедшее значение есть в списке — оставить, иначе подставить `другое`. Два уровня, и оба нужны: схема говорит модели, что от неё хотят, сверка проверяет, что она это сделала.**

# Что видит человек




**Слева вход: форма объясняет, что произойдёт с заявкой, прежде чем её отправят. Справа выход: все четыре шага, а не только ответ — что пришло, что предложила модель, что осталось после справочника, куда легла строка.**

**Промежуточные шаги печатаются намеренно: иначе ошибку видно только там, где она уже стала результатом. И выход показывают на том же экране, где был вход, — переключение во вкладку «посмотреть, сработало ли» есть главный способ не заметить, что сработало неправильно.**

# Когда закрытый список нужен

| ЗАКРЫТЫЙ СПИСОК НУЖЕН | ЗАКРЫТЫЙ СПИСОК НЕ НУЖЕН |
| --- | --- |
| результат шага читает следующий шаг: по нему фильтруют, сортируют, считают или маршрутизируют | результат читает человек и он же решает, что с ним делать — свободный текст ему помогает, а не мешает |
| по полю будет отчёт: «сколько заявок про оплату за месяц» | поле нужно один раз, для одного письма, и больше его никто не откроет |
| за результат надо отвечать: он уходит в начисление, рассылку, обязательство | ошибку заметит тот же человек в тот же день, и исправит её руками за минуту |
| поток растёт и типов входа становится больше одного | двадцать обращений в неделю, пять минут в день, и так будет дальше |

**Предел, после которого список перестаёт работать: в нём нет «иначе». Список без значения «другое» и «не указано» не отказывает — он заставляет модель выбрать ближайшее. Вы получите законное значение на заявке, которой в вашем списке не место, и не узнаете об этом никогда.**

**Первая строка оси закрыта. Три узла заявку довозят. Тема при этом будет какой угодно, срочности не будет вовсе — ручаться нечем. Разница между «работает» и «можно ручаться» здесь ровно в двух узлах: тот, что сверяет со списком, и тот, что показывает результат.**

# Кейс 2. Заявки с несколькими темами

*«В заявке две темы, а в таблице одна.»*

# Вторник: половина заявки обслужена

**Процесс работает неделю. Ольга к нему привыкла: смотрит таблицу утром, звонит по срочным, остальное раскладывает после обеда. Жалоб нет.**

1. Вторник, 11:20. Приходит заявка: «можно перенести занятие со вторника? И подскажите, как оплатить».
2. В таблице появляется строка: тема «перенос занятия», срочность «не указано». Всё заполнено, значение из списка, прогон зелёный.
3. Ольга видит тему, переносит занятие, закрывает заявку. Делает ровно то, что написано в столбце.
4. Пятница. Человек звонит сам: «а со счётом что?» Про оплату не узнал никто — ни таблица, ни журнал, ни экран результата о ней не сказали.

**Это не ошибка процесса. Мы обещали, что клетка не будет пустой и значение будет из списка, — обещание выполнено. Что в клетке окажется всё, что было в заявке, мы не обещали. Беда в том, что граница наших обещаний обнаружилась не на разборе, а в разговоре с клиентом — и ни один сторож не мог сработать, потому что формально всё в порядке.**

# Как корректируем устройство процесса

**Как корректируем устройство процесса?**

- писать первую из найденных
- писать «не определено»
- две строки из одного письма
- расширять справочник всеми сочетаниями тем

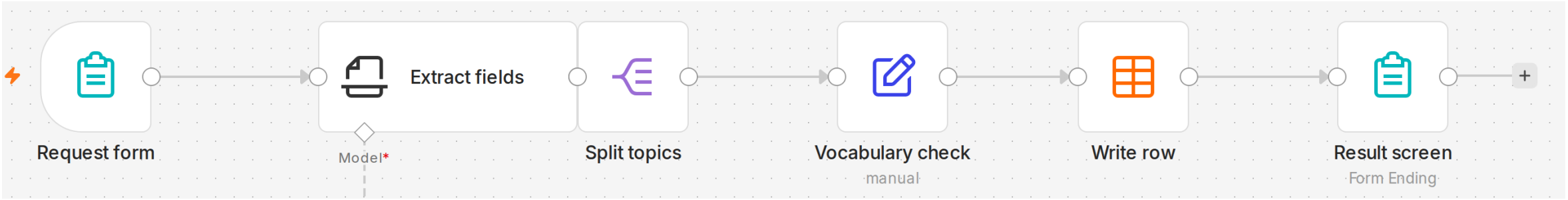
# Что делает каждый способ

| СПОСОБ | ЧТО ДЕЛАЕТ | ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ |
|---|---|---|
| **первую из найденных** | ничего не меняет в процессе и не стоит ни минуты работы | вторая тема не остаётся нигде: ни в таблице, ни в журнале, ни на экране. Через три дня о ней напомнит человек — это ровно то, что мы видели |
| **«не определено»** | честно говорит «автоматика не справилась»: заявка видна и ждёт человека | в «не определено» валится и многотемье, и настоящая экзотика — очередь растёт, и уже не видно, что именно непонятно |
| **две строки из одного письма** | каждая тема получает свой маршрут и своего ответственного, закрывается независимо, сортировка по теме не ломается | одна заявка превращается в две записи: надо решить, что считать выполненным, и не звонить человеку дважды |
| **справочник всеми сочетаниями** | формально клетка остаётся одна, и ничего в процессе менять не надо | число значений растёт как число сочетаний: четыре темы дают одиннадцать пар и троек. Сортировать по такому столбцу нельзя, и каждая новая тема удваивает справочник |

**Поворот. Первый и второй различаются не точностью, а знанием о своём незнании. Первый уверен и неправ. Второй не знает — и говорит об этом вслух, в столбце. Второе дороже в разработке и дешевле в эксплуатации.**

**Берём третий. Две строки — единственный способ, при котором обе темы получают маршрут и срок. Четвёртый ломается на арифметике сочетаний, второй годится, если очередь «не определено» кто-то разбирает, первый — молчаливая потеря.**

# Что меняем в процессе




**Форма, модель, сверка, запись и экран результата остаются ровно такими, какими были. Новый узел стоит сразу после извлечения: Split topics.**

**Узел записи не меняется вовсе — и всё-таки пишет две строки вместо одной. Почему, разберём на следующем экране: это не свойство узла, а свойство того, как в n8n ходят данные.**

# Как расщепляется запись




**Модель данных. Каждый элемент — объект с обязательной обёрткой `json`, поэтому в выражениях везде `$json.поле`. В кейсе 1 список всегда состоял из одного элемента, и это было незаметно.**

**Что делает узел расщепления. Берёт поле-массив — у нас это список тем — и превращает один элемент со списком из двух тем в два элемента по одной теме. Остальные поля копируются в оба.**

**Дальше не меняется ничего. Сверка срабатывает дважды, запись срабатывает дважды — не потому, что их переписали, а потому, что элементов стало два. Отсюда же известная беда: пять писем вместо одного, когда узел отправки сработал на каждом элементе и ни у кого не спросил.**

# Что получилось в таблице

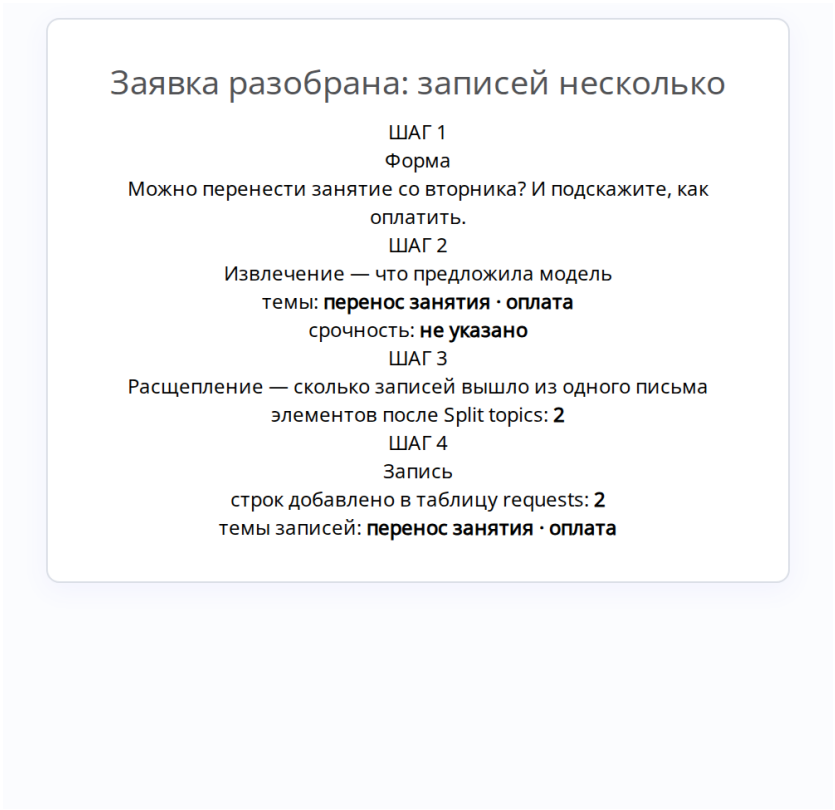
| id | received_at | name | contact | request_text | topic | urgency | |
|---|---|---|---|---|---|---|---|
| **одно письмо Анны — две строки:** | | | | | | | |
| 19 | 2026-10-15T10:15:40.859+03:00 | Анна | anna@example.com | Можно перенести занятие со вторни | перенос занятия | не указано | *Em* |
| 20 | 2026-10-15T10:15:40.873+03:00 | Анна | anna@example.com | Можно перенести занятие со вторни | оплата | не указано | *Em* |

**Строки 19 и 20: тот же текст, тот же контакт, та же секунда. Разные темы — перенос занятия и оплата.**

**Что это даёт Ольге. Переносом занимается расписание, оплатой бухгалтерия. Каждая строка закрывается своим человеком и в свой срок; ни одна не ждёт, пока разберутся с другой.**

**Чем платим. Одна заявка — две записи. Нужно решить, что считать выполненной заявкой, и не позвонить человеку дважды по одному письму. Это вопрос не к процессу, а к договорённости в команде, и решить его придётся до запуска.**

# Что видит человек




**Счётчик — проверка, а не украшение. «Тем две, а строка одна» видно сразу, без журнала.**

**Если шаг размножает или схлопывает записи — покажите число до и после. Тихая потеря половины данных выглядит как успешная работа.**

# Когда запись расщепляют

| РАСЩЕПЛЯТЬ НУЖНО | РАСЩЕПЛЯТЬ НЕ НУЖНО |
| --- | --- |
| у частей разные исполнители или разные сроки — перенос в расписание, оплата в бухгалтерию | части делает один человек за один раз: дробление только добавит ему работы |
| по этому столбцу сортируют или маршрутизируют, и смешанное значение ломает сортировку | столбец читает глазами человек — он поймёт «перенос и оплата» в одной клетке |
| части закрываются независимо, и одна не должна ждать другую | заявка имеет смысл только целиком: половина отгрузки — не отгрузка |
| потеря части дороже, чем двойной учёт | двойной звонок клиенту дороже, чем задержка по второй теме |

**Предел: учёт без договорённости. Расщепление создаёт вопрос «что считать выполненной заявкой», и ответ на него процесс дать не может — его даёт команда. Две строки без такой договорённости хуже одной: по ним нельзя ни отчитаться, ни понять, кто кому звонит.**

**Вторая строка оси закрыта. Клетка заполнена, значение законное — и это выглядит успехом. Чем оказалось: в заявке было две темы, а записали одну, и вторая не осталась нигде — ни в таблице, ни в журнале, ни на экране.**

# Кейс 3. Отказ модели

*Пятница заполнена, понедельник заполнен, выходных в таблице нет вовсе.*

# Что произошло за выходные

**Процесс работает три недели. Ольга смотрит таблицу утром. В пятницу вечером всё было как обычно.**

1. Понедельник, 9:00. Ольга открывает таблицу — по другому делу, посмотреть вчерашние заявки.
2. Видит: строки за пятницу есть, строки за понедельник есть. **За субботу и воскресенье — ни одной.**
3. Письма об ошибке никто не получал. И это ничего не значит: кому его отправлять, никто не настраивал.
4. Процесс всё это время был опубликован и включён. Падал ли он на каждой заявке, на части или писал не туда — неизвестно.

**Ни один сторож не сработал, и не мог. Сторожа срабатывают на ошибки, а здесь нет ошибок — здесь нет записей. На отсутствие чего-то сработать нечем. Заметили случайно: Ольга зашла в таблицу по другому поводу. Сколько бы это продолжалось иначе — тоже неизвестно.**

# Поведение узла при неудачном вызове

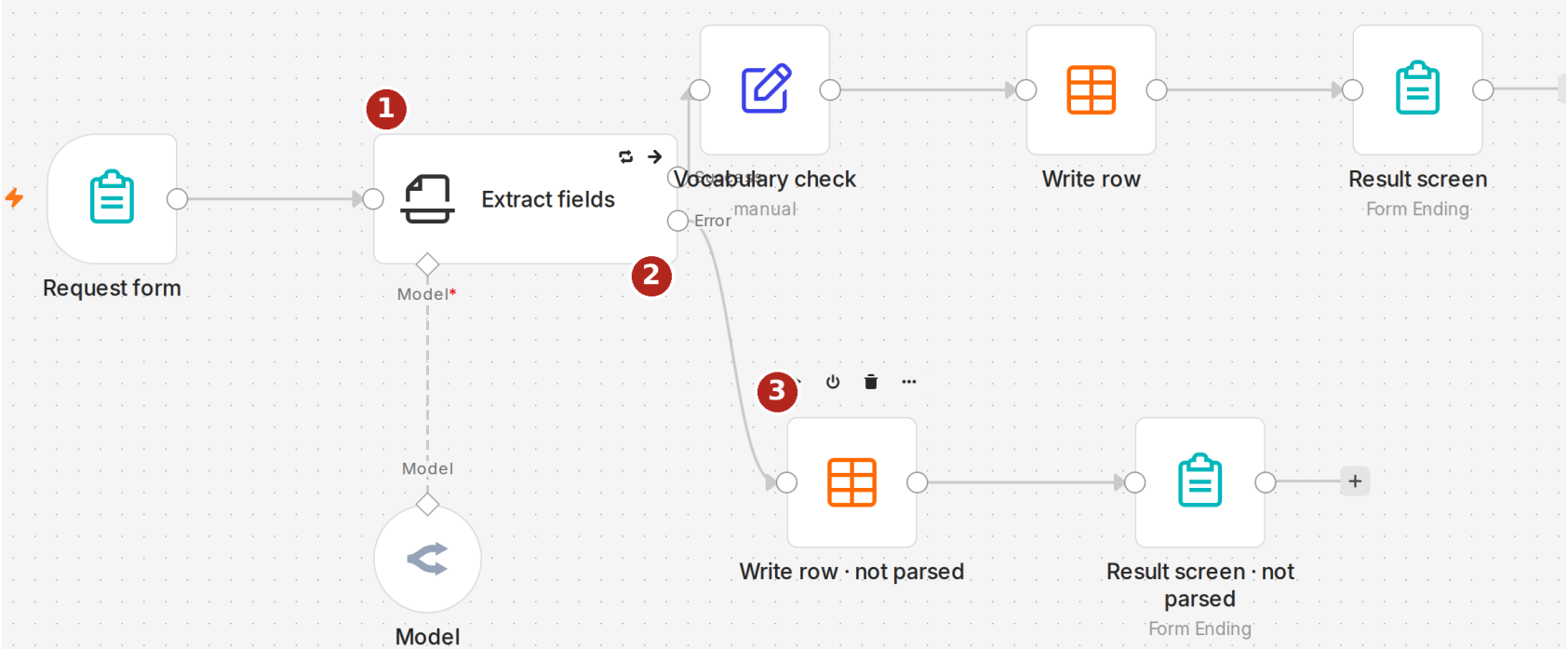
**Что должен делать узел, когда вызов не удался?**

- **остановить весь процесс**
- **пропустить и идти дальше**
- **повторить**
- **отдельная ветка**

# Что делает каждая настройка

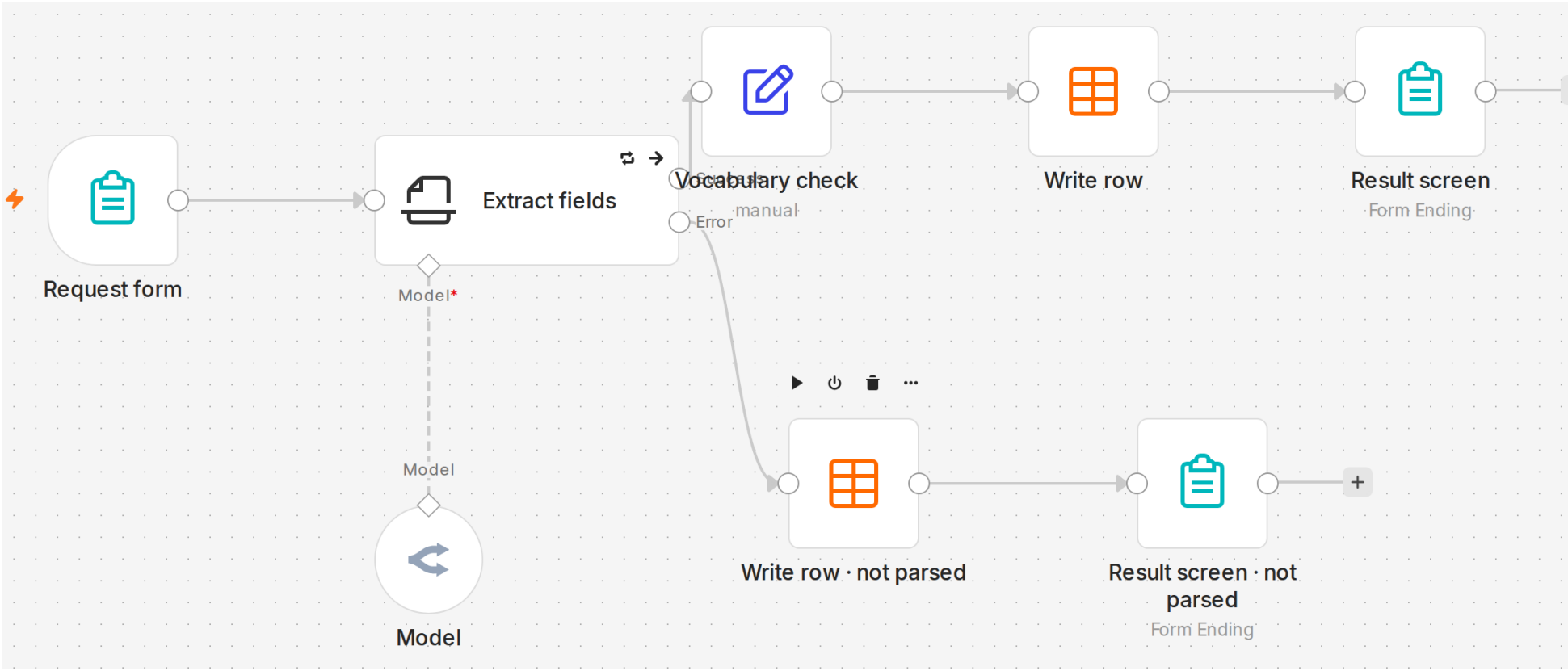
| НАСТРОЙКА УЗЛА | ПОВЕДЕНИЕ | КОГДА УМЕСТНО |
|---|---|---|
| **On Error = Stop Workflow** (по умолчанию) | прогон падает целиком, дальше ничего не идёт | следующий шаг необратим и дорог: счёт, письмо клиенту, платёж |
| On Error = Continue | текст ошибки идёт дальше **как обычные данные** | почти никогда: ошибка молча доедет до таблицы |
| **On Error = Continue (using error output)** | элемент уходит в отдельный выход `error` | разбирать отказы своей веткой |
| **Retry On Fail** | повтор N раз, затем одно из трёх выше | сетевые отказы, таймауты, лимиты провайдера |

# Процесс с веткой отказа




**Заметьте: повтор виден снаружи, значком на самом узле. Большинство настроек так не видно — за ними надо открывать карточку.**

# Повтор попыток и ветка отказа




**ВО ВКЛАДКЕ НАСТРОЕК УЗЛА ИЗВЛЕЧЕНИЯ**

| | |
|---|---|
| Retry On Fail · On Error | 3 попытки, пауза 2 с → `Continue (using error output)` |

# Когда повторять, а когда нет

**ПОВТОРЯТЬ НУЖНО**

отказ временный: таймаут, превышение лимита запросов, «сервис перегружен» — со второй попытки обычно проходит

операция идемпотентна: её можно выполнить дважды с тем же результатом — чтение, пересчёт, идемпотентная запись по ключу

повтор стоит здесь и только здесь, в одном месте стека

задержки с разбросом, и общее число попыток ограничено

**ПОВТОРЯТЬ НЕЛЬЗЯ**

отказ постоянный: неверный ключ, негодный запрос, нет обязательного поля. Повтор лишь утроит расход на ту же ошибку

у операции есть последствия: отправить письмо, создать заказ, списать деньги. Таймаут не говорит, произошло ли действие

повтор уже стоит слоем ниже или выше: слои множат друг друга

задержка фиксированная: все клиенты повторят синхронно и добьют сервис, который начал подниматься

**Три попытки на каждом из пяти слоёв дают 243-кратную нагрузку на нижний. Поэтому повторяют в одном месте — и там, где знают, что повторять безопасно.**

**Предел: отказ, которому некуда идти. Такой отказ не исчезает — он превращается в отсутствие записи, а отсутствие записи не мониторит никто: ни один сторож не умеет сработать на то, чего нет. Отсюда и двое суток пропуска.**

**Третья строка оси закрыта. Прогон зелёный, в журнале ни одной ошибки — выглядит успехом. Чем оказалось: три отказа подряд и заявки за выходные неразобранные.**

# Кейс 4. Тема определена неверно

*«Тут написано „курс“, а человек про счёт спрашивал. И так раза три за день.»*

# Что значит «тема не та»

**Заявка: «Подскажите, сколько стоит обучение для группы из восьми человек, и можно ли выставить счёт на ООО?»**

**Процесс записал: тема `курс`, срочность не `указано`.**

**На самом деле: человеку нужен счёт. Это `оплата`, и заявка должна была уйти в бухгалтерию.**

1. Модель увидела в первом предложении «обучение» и «стоит» — и выбрала `курс`. Значение законное, из справочника.
2. Заявка легла в список к тому, кто обзванивает интересующихся курсами.
3. Он звонит, выясняет, что нужен счёт, и передаёт в бухгалтерию. Это та самая лишняя передача из кейса 1 — день.
4. Ольга замечает ошибку в таблице глазами. **И не может никуда её записать:** столбца для этого нет.

**Три наших сторожа молчат, и все три правы. Значение не пустое, значение из справочника, прогон зелёный. Мы научились проверять форму ответа — и ни разу не проверили его смысл. Хуже другое: пока ошибка нигде не отмечена, мы не знаем даже, как часто она случается.**

# Как поднять качество распознавания

**Как сделать, чтобы тема определялась верно чаще?**

**усилить запрос**

**вторая модель проверяет первую**

**алгоритмические правила поверх**

**модель помощнее**

# Что даёт каждый способ

| СПОСОБ | ЧТО ДАЁТ | ЧЕМ ПЛАТИМ |
|---|---|---|
| **усилить запрос** | **+5–12 п.п.** от примеров: 0,35 без примеров → 0,41 на двенадцати, лучшая сборка 0,48 | час работы один раз; запрос становится длиннее, и его надо держать в порядке |
| **вторая модель проверяет первую** | **0,57 → 0,61** у разбора четырьмя запросами с голосованием; ловит разовые срывы | каждый прогон вдвое дороже и вдвое дольше; расхождение двух моделей надо как-то разрешать |
| **алгоритмические правила поверх** | **+2–3 п.п.** точно и предсказуемо: «есть слово „счёт“ или „ООО“ — тема `оплата`» | правил становится много, они начинают противоречить друг другу, и живой язык всё равно шире |
| **модель помощнее** | обычно заметно лучше на длинном и запутанном тексте | платите на **каждом** прогоне, навсегда; на коротких заявках разница часто в пределах шума |

**Поворот. Первые три стоят один раз, четвёртый — каждый день. Поэтому порядок почти всегда такой: сначала запрос, потом правила на узкие случаи, потом вторая модель там, где цена ошибки высока, и только потом более дорогая модель.**

**И то, без чего всё перечисленное бессмысленно: ни один из четырёх способов нельзя оценить, пока ошибки нигде не отмечаются. «Стало лучше» без числа — это вера.**

# Решение: усиленный запрос и отметка ошибки

**Описания категорий словами. Было: «о чём заявка». Стало: «`оплата` — человек спрашивает про счёт, реквизиты, безналичный расчёт или возврат денег, даже если в той же фразе упомянут курс». Именно эта оговорка чинит нашу заявку.**

**Примеры заявок прямо в запросе. Три-пять настоящих коротких заявок с правильным ответом, и обязательно — трудные, а не очевидные: та самая, про счёт на ООО. Замеренный прирост даёт не количество примеров, а их трудность.**

**Правило воздержания. «Если подходит больше одной темы и ни одна не очевидна — ставь `не определено`». Законный отказ из кейса 2, только теперь он включается не по пустоте, а по неуверенности.**

**Столбец `topic_wrong` в таблице. Один флажок, который Ольга ставит, увидев неверную тему. С этого дня у вас есть доля ошибок, и любое следующее улучшение — измеримо. Это и есть та размеченная история, которой у центра нет: через месяц она появится сама.**

# Чем поднимать качество

### ХВАТИТ УСИЛЕНИЯ ЗАПРОСА

ошибки разные и каждая объяснима словами: модель не знала, что считать оплатой

вы можете назвать правильный ответ для каждой разобранной ошибки

заявки короткие и однородные — на них разница между моделями в пределах шума

у вас нет размеченной истории — запрос работает без неё

### НУЖНО ЧТО-ТО СВЕРХ ЗАПРОСА

одна и та же ошибка повторяется дословно — дешевле поставить правило, чем уговаривать модель

цена одной ошибки высока: деньги, здоровье, юридические последствия — тогда вторая проверка окупается

текст длинный и запутанный: здесь более сильная модель даёт заметный прирост, и за него есть смысл платить

история накопилась: тысячи заявок с правильными ответами делают дообучение осмысленным

**Предел: улучшение без замера. Пока ошибки нигде не отмечаются, любой из четырёх способов даёт только ощущение. Сначала столбец отметки — потом улучшения; иначе вы будете честно работать и не сможете сказать, помогло ли.**

**Четвёртая строка оси закрыта. Значение из справочника, прогон зелёный — и это выглядит успехом. Чем оказалось: тема неверная, и узнать об этом неоткуда — все наши сторожа проверяют форму, а не смысл.**

# Черновик и опубликованная версия




1 значок Published — в бою работает ОПУБЛИКОВАННАЯ версия, а не та, что открыта в редакторе. Правка на холсте сохраняется в черновик сама; в бой она уходит отдельным действием.

**Правки идут в черновик постоянно и сами. Кнопки «Сохранить» в интерфейсе нет вовсе — и именно поэтому возникает уверенность, что всё уже в деле.**

**В бой правка уходит отдельным действием — Publish. Боевые прогоны исполняют опубликованную версию, а ручной запуск — черновик. Отсюда вся путаница: вы правите, запускаете руками, видите правильный результат — и он честный, просто не тот, что работает в бою. Слова «активировать» в n8n 2.x больше нет.**

**Что проверяется только настоящим событием: обработчик ошибок (его триггер при ручном запуске не срабатывает вовсе) · расписание · обращение к чужой системе · поведение на объёме. Руками проверяются шаги, выражения и форма — этого достаточно для логики и недостаточно для боя.**

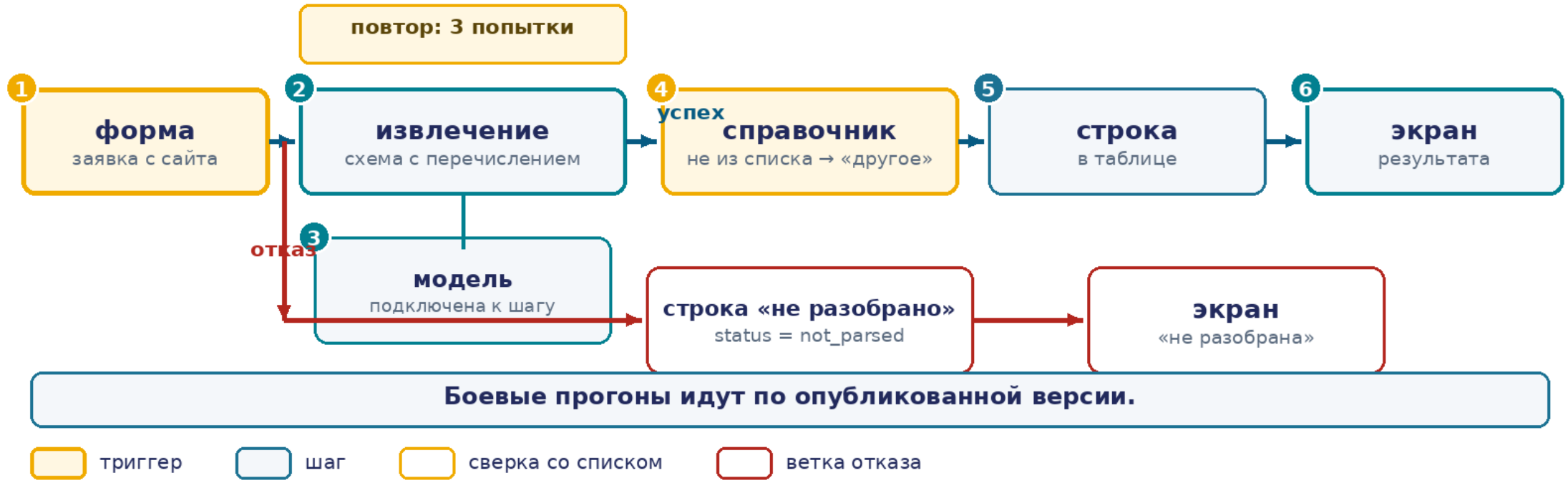
**Перед тем как отдать файл сценария — прочитайте его глазами. Ключей в нём нет, их инструмент держит отдельно. А имена подключений, адреса, пути, названия таблиц и внутренних сервисов — есть. Это тридцать секунд.**

# Четыре строки оси: что оказалось

| ВЫГЛЯДИТ РАБОЧИМ | ЧЕМ ОКАЗАЛСЯ |
|---|---|
| три узла, заявка доезжает до таблицы | тема какая угодно, срочности нет вовсе, ручаться нечем |
| клетка заполнена, значение законное | вторая тема заявки исчезла молча |
| прогон зелёный, в журнале ни одной ошибки | три отказа подряд, заявки за выходные не разобраны |
| значение из справочника, прогон зелёный | тема неверная, и узнать об этом неоткуда |

**Процесс работает — или за него можно ручаться? Четыре строки справа и есть четыре способа это проверить, и ни один не требует верить на слово.**

# Что уносим с занятия




| ФАЙЛ | STUDENT.JSON |
|---|---|
| **как поставить** | меню сценария → Import from File → выбрать файл |
| **что нужно** | свой n8n и ключ провайдера модели (инструкция) |
| **что делать** | прогнать на своих трёх заявках и посмотреть, где развалится |
| **что завести своё** | учётные данные модели и свою таблицу: в файле они пустые намеренно |

# Что дальше

**Сейчас: любая заявка → один и тот же шаг → таблица**

**Дальше: заявка про оплату → одно · про перенос → другое · непонятная → третье**

**Семинар 8: развилка и как она устроена · что делать с тем, что не подошло ни под одну ветку · проверка результата шага до того, как он уехал дальше · и почему ветки, которые выглядят параллельными, выполняются по очереди**